MERCHANT PRODUCT MANAGEMENT SYSTEM USING HIBERNATE ORM

INDEX

S.NO	TITLE	PAGE NUMBER
1.	INTRODUCTION	2
2.	PROBLEM STATEMENT	2-3
3.	OBJECTIVES OF THE PROJECT	3
4.	SCOPE OF THE PROJECT	4-5
5.	TECHNOLOGIES USED	5-7
6.	ER DIAGRAM	8
7.	IMPLEMENTATION AND OUTPUT	9-26
8.	SUMMARY	26

1. INTRODUCTION

The **Merchant Product Management System using Hibernate & MySQL** is a Java-based application designed to manage merchant details and their product information in an efficient and systematic manner. In many small and medium businesses, merchant and product records are often maintained manually, which may lead to data redundancy, loss of records, and difficulty in searching or updating information. This project provides a computerized solution to overcome such issues.

The main purpose of this system is to store merchant details such as name, phone number, GST number, email, and password in a secure database. After successful registration, merchants can perform various product-related operations such as adding new products, updating existing products, and searching products based on brand, category, or merchant ID.

This project is developed using **Java**, **Hibernate ORM**, and **MySQL** database. Hibernate is used as an Object Relational Mapping (ORM) framework that simplifies database interaction by mapping Java classes to database tables. It reduces the need for writing complex SQL queries manually and improves application maintainability.

The system mainly consists of two modules:

1. **Merchant Module**
   This module is responsible for merchant registration, updating merchant details, finding merchant records by ID, and verifying merchant login using email/password or phone/password.
2. **Product Module**
   This module allows merchants to add products, update product details, and search products based on different criteria such as brand, category, and merchant ownership.

2. PROBLEM STATEMENT

In today's business environment, merchants handle a large number of products and customer-related operations daily. Maintaining merchant details and product records manually through notebooks, paper files, or spreadsheets is time-consuming and inefficient. Manual systems often lead to data duplication, calculation errors, missing records, and difficulty in searching or updating information when required.

Many small and medium-scale businesses do not have a proper digital system to manage merchant registration, login verification, and product information. As the number of merchants and products increases, it becomes difficult to organize data properly. Searching products by brand, category, or merchant ownership also becomes a complex task in manual record systems.

Another major issue is data security. Merchant login details such as email, phone number, and password must be stored securely. In traditional systems, these details may be exposed or lost due to improper maintenance. There is also no efficient way to verify merchant credentials instantly.

To overcome these problems, a computerized system is required that can store merchant and product details in a centralized database. The system should provide features such as merchant registration, updating records, login verification, product addition, product modification, and fast product search operations.

The **Merchant Product Management System using Hibernate & MySQL** is developed as a solution to these challenges. It automates business operations, improves data accuracy, reduces paperwork, saves time, and provides an easy method to manage merchants and their products efficiently.

3. OBJECTIVES OF THE PROJECT

The main objective of the **Merchant Product Management System using Hibernate & MySQL** is to develop an efficient and user-friendly application for managing merchant details and their products digitally. The system is designed to replace manual record-keeping methods with an automated database management solution.

This project aims to store merchant information such as name, phone number, GST number, email, and password in a secure and organized manner. It also allows merchants to manage their product details such as product name, brand, category, and cost.

Another important objective of this project is to provide easy and fast search operations. Users can search products by merchant ID, brand name, or category. This saves time and improves productivity in business operations.

The project also focuses on implementing secure login verification using email/password or phone/password. This ensures that only authorized merchants can access the system.

By using **Hibernate ORM**, the project aims to simplify database operations such as insert, update, delete, and fetch records through Java objects instead of writing complex SQL queries manually.

Specific Objectives

1. To develop a computerized system for merchant and product management.
2. To store merchant records securely in a MySQL database.
3. To add, update, and retrieve product information easily.
4. To verify merchant login credentials securely.
5. To provide search functionality based on brand, category, and merchant ID.
6. To reduce manual paperwork and human errors.
7. To improve speed, accuracy, and efficiency of data management.
8. To understand practical implementation of Java, Hibernate, and MySQL integration.

4. SCOPE OF THE PROJECT

The scope of the **Merchant Product Management System using Hibernate & MySQL** is to provide a digital platform for managing merchant information and product details in an efficient and organized manner. This system is useful for businesses that need to maintain merchant records and handle multiple products with proper database management.

The project mainly focuses on two important modules: Merchant Management and Product Management. In the Merchant module, users can register new merchants, update merchant details, search merchant records by ID, and verify login credentials using email/password or phone/password. In the Product module, users can add new products, update product details, and search products by merchant ID, brand, or category.

This system can be implemented in small shops, retail stores, wholesale businesses, agencies, supermarkets, and product distribution centers where merchant and product records need to be maintained regularly. It helps in reducing manual paperwork and provides quick access to data whenever required.

The project also demonstrates the practical use of Hibernate ORM for object-relational mapping and MySQL for database storage. It provides learning opportunities for students and developers to understand Java-based enterprise application development.

Areas Covered in the Project

1. Merchant registration and management
2. Merchant login verification
3. Product addition and updation
4. Product searching and retrieval
5. Database connectivity using Hibernate
6. CRUD operations using Java
7. One-to-Many relationship between Merchant and Product

Limitations of Current Scope

1. The project is console-based and does not include a graphical user interface.
2. It supports basic merchant operations only.
3. No payment or billing system is included.
4. No online ordering feature is available.

Future Scope

The system can be expanded in the future by adding:

- Web-based user interface
- Admin dashboard
- Order management system

- Payment gateway integration
- Product stock management
- Report generation
- Cloud database support

5. TECHNOLOGIES USED

The **Merchant Product Management System** is developed using modern technologies that help in creating a reliable, secure, and efficient application. Each technology used in this project plays an important role in application development, database connectivity, and data management.

This chapter explains the technologies used for implementing the project.

5.1 Java

Java is the core programming language used to develop this project. It is one of the most popular object-oriented programming languages and is widely used for developing desktop, web, and enterprise applications.

Java provides features such as:

- Platform independence
- Object-Oriented Programming concepts
- Security
- Robust exception handling
- Multithreading support
- Rich API libraries

In this project, Java is used to create classes such as Merchant, Product, DAO classes, and Controller classes.

5.2 Hibernate ORM

Hibernate is an Object Relational Mapping (ORM) framework used to simplify database operations in Java applications.

Hibernate maps Java classes to database tables automatically. Instead of writing SQL queries manually, developers can perform database operations using Java objects.

Advantages of Hibernate:

- Reduces SQL coding
- Automatic table mapping
- Easy CRUD operations

- Supports relationships (OneToMany, ManyToOne)
- Better performance with caching
- Database independent

In this project, Hibernate is used to save, update, fetch, and verify merchant and product records.

5.3 MySQL

MySQL is the relational database management system used to store project data permanently.

It stores merchant details and product details in tables.

MySQL Features:

- Fast and reliable
- Open-source database
- Secure data storage
- Supports SQL queries
- Multi-user support

In this project, MySQL contains two main tables:

1. Merchant Table
2. Product Table

5.4 JDBC

JDBC (Java Database Connectivity) is used internally by Hibernate to connect Java applications with MySQL database.

It helps in:

- Opening database connection
- Executing queries
- Managing transactions
- Closing connections safely

5.5 Maven

Maven is a build automation and dependency management tool.

It is used to manage required libraries such as:

- Hibernate Core
- MySQL Connector
- JPA API

Maven uses pom.xml file for dependency configuration.

5.6 IDE Used

An **Integrated Development Environment (IDE)** is used for coding and running the project.

Recommended IDEs:

- Eclipse IDE
- IntelliJ IDEA
- NetBeans

IDE helps in:

- Writing code
- Debugging errors
- Managing project files
- Running Java programs

5.7 XML Configuration Files

The project uses:

hibernate.cfg.xml

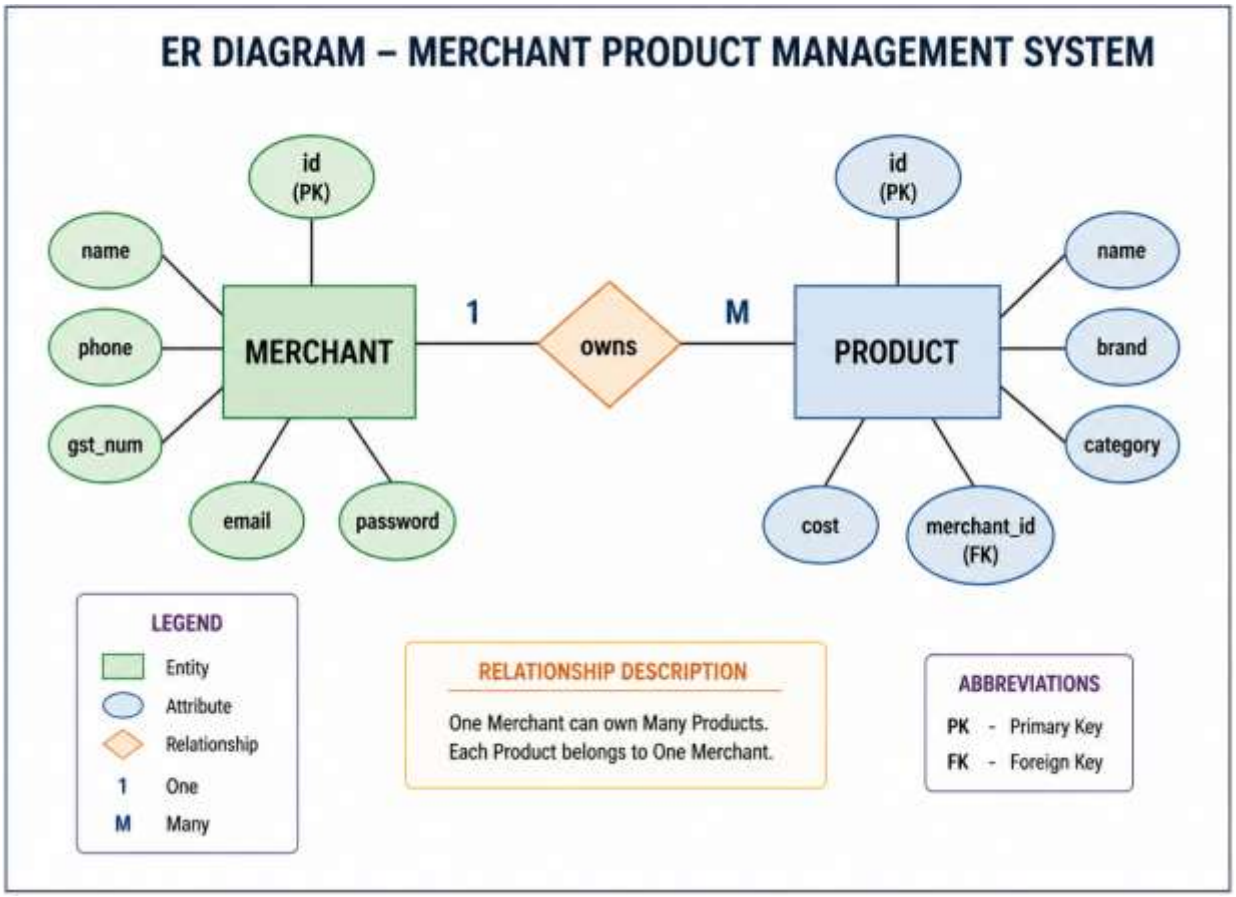
Used for:

- Database URL
- Username
- Password
- Hibernate dialect
- Mapping classes

persistence.xml

Used for JPA configuration.

6. ER DIAGRAM



The above image represents the Entity Relationship (ER) Diagram of the Merchant Product Management System. It shows the structure of the database and the relationship between the two main entities, Merchant and Product. The Merchant entity contains details such as id, name, phone, gst number, email, and password, which are used to store merchant information. The Product entity contains attributes such as id, name, brand, category, cost, and merchant_id, which are used to store product details. In the diagram, the relationship named “owns” connects Merchant and Product, where one merchant can own many products, while each product belongs to only one merchant. The symbol 1 near Merchant and M near Product indicates a One-to-Many relationship. The primary key (PK) uniquely identifies each record, while the foreign key (FK) in the Product table links products with the merchant. This ER Diagram helps in understanding the database design clearly and is useful for implementing the project using Hibernate and MySQL.

7. IMPLEMENTATION AND OUTPUT

pom.xml

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSc
  <modelVersion>4.0.0</modelVersion>
  <groupId>org.jsp</groupId>
  <artifactId>merchant-product-app</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <dependencies>

        <!-- Source: https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.6.15.Final</version>
    <scope>compile</scope>
</dependency>

<!-- Source: https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>5.1.6</version>
    <scope>compile</scope>
</dependency>

  </dependencies>
</project>
```

persistence.xml

```
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd (xsi:schemaLocation with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
                                 http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd" version="2.1">
    <persistence-unit name="dev">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <properties>
                        <!-- Database Properties -->
            <property name="javax.persistence.jdbc.driver" value="com.mysql.jdbc.Driver"/>
            <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/merchantappdb?createDatabaseIfNotExist=true"/>
            <property name="javax.persistence.jdbc.user" value="root"/>
            <property name="javax.persistence.jdbc.password" value="suman"/>

                        <!-- ORM Tools Properties -->
            <property name="hibernate.dialect" value="org.hibernate.dialect.MySQL8Dialect"/>
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>
```

Merchant.java

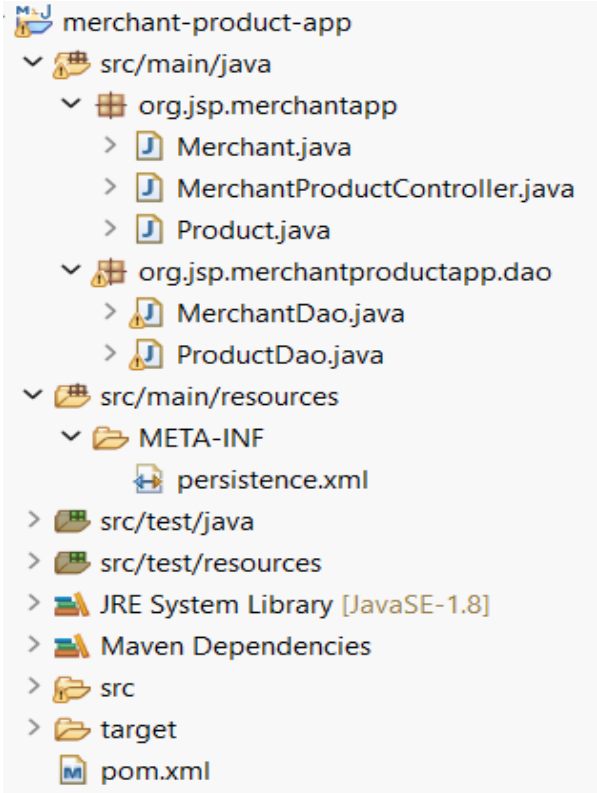
```java
import java.util.List;

@Entity
public class Merchant {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private int id;
    private String name;
    private long phone;
    private String gst_num;
    private String email;
    private String password;

    @OneToMany(mappedBy = "merchant")
    private List<Product> products;

    // getter and setter
    // toString()
}
```

Product.java

```java
import javax.persistence.*;

@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private String brand;
    private String category;
    private double cost;

    @JoinColumn
    @ManyToOne(cascade = CascadeType.ALL)
    private Merchant merchant;

    // getter and setter
    // toString()
}
```

MerchantProductController.java

```java
public class MerchantProductController {
    static ProductDao pdao = new ProductDao();
    static MerchantDao mdao = new MerchantDao();
    static Scanner sc = new Scanner(System.in);
    public static void main(String[] args) {
        System.out.println("1. Save Merchant");
        System.out.println("2. Update Merchant");
        System.out.println("3. Find Merchant by Id");
        System.out.println("4. Verify Merchant by Email and Password");
        System.out.println("5. Verify Merchant by Phone and Password");
        System.out.println("6. Add Product");
        System.out.println("7. Update Product");
        System.out.println("8. Find Products by Merchant id");
        System.out.println("9. Find products by brand");
        System.out.println("10. Find products by Category");
        System.out.println("Enter Your choice??");

        int choice = sc.nextInt();
        switch (choice) {
        case 1:saveMerchant();
            break;
        case 2:updateMerchant();
            break;
        case 3:findMerchantById();
            break;
        case 4:verifyMerchantByEmailAndPassword();
            break;
        case 5:verifyMerchantByPhoneAndPassword();
            break;
        case 6:addProduct();
            break;
        case 7:updateProduct();
            break;
        case 8:findProductByMerchantId();
            break;
        case 9:findProductByBrand();
            break;
        case 10:findProductByCategory();
            break;
        default: System.err.println("Guldu!!!, Open your eyes and see only 10 Choices are there");
            break;
        }
```



Case 1 : Save Merchant

MerchantDao.java

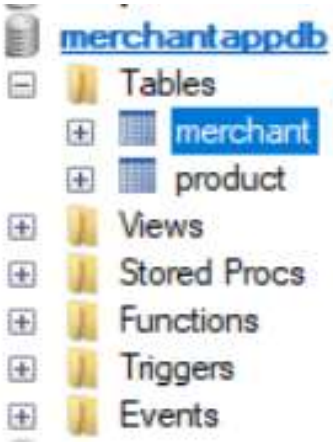
```
import javax.persistence.*;
import org.jsp.merchantapp.Merchant;

public class MerchantDao {
    EntityManagerFactory fac = Persistence.createEntityManagerFactory("dev");
    EntityManager man = fac.createEntityManager();
    private Merchant mdb;

    public Merchant saveMerchant(Merchant m) {
        EntityTransaction tran = man.getTransaction();
        tran.begin();
        man.persist(m);
        tran.commit();
        return m;
    }
```

Output :

```
1. Save Merchant
2. Update Merchant
3. Find Merchant by Id
4. Verify Merchant by Email and Password
5. Verify Merchant by Phone and Password
6. Add Product
7. Update Product
8. Find Products by Merchant id
9. Find products by brand
10. Find products by Category                    -- name, phone, gst, email, password");
Enter Your choice??
1
```

```
    String gst = sc.next();
    String email = sc.next();
    String password = sc.next();

    Merchant m = new Merchant();
    m.setName(name);
    m.setPhone(ph);
    m.setGst_num(gst);
    m.setEmail(email);
    m.setPassword(password);

    Merchant mdb = mdao.saveMerchant(m);
    System.out.println("Merchant is saved with an id " + mdb.getId());
}
```

```
Enter Merchant info ---- name, phone, gst, email, password
Suman
8855336622
SUM12334
suman@gmail.com
suman12345
Hibernate:
    insert
    into
        Merchant
        (email, gst_num, name, password, phone)
    values
        (?, ?, ?, ?, ?)
Merchant is saved with an id 3
```



Case 2 : Update Merchant

MerchantProductController.java

```java
private static void updateMerchant() {
    System.out.println("Enter Merchant Info to Update... id, name, phone, gst, email, password");
    Merchant m = new Merchant();
    m.setId(sc.nextInt());
    m.setName(sc.next());
    m.setPhone(sc.nextLong());
    m.setGst_num(sc.next());
    m.setEmail(sc.next());
    m.setPassword(sc.next());
    Merchant mdb = mdao.updateMerchant(m);
    if(mdb != null)
    {
        System.out.println("Merchant info is updated!!");
    }
    else {
        System.out.println("Merchant info is not updated!");
    }
}
```

MerchantDao.java

```java
public Merchant updateMerchant(Merchant m) {
    EntityTransaction tran = man.getTransaction();
    tran.begin();
    Merchant mdb = man.find(Merchant.class, m.getId());
    if(mdb != null)
    {
        mdb.setName(m.getName());
        mdb.setPhone(m.getPhone());
        mdb.setGst_num(m.getGst_num());
        mdb.setEmail(m.getEmail());
        mdb.setPassword(m.getPassword());
        tran.commit();
        return m;
    }
    else {
        return null;
    }
}
```

Before Update :

Query | Table Data

id	email	gst_num	name	password	phone
1	ganesh@gmail.com	GKM12345	Ganesh	ganesh12345	8855221144
2	mehta@gmail.com	MEH12344	Mehta	mehta1234	2587410345
3	suman@gmail.com	SUM12334	Suman	suman12345	8855336622
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Output :

```
Enter Your choice??
2
Enter Merchant Info to Update... id, name, phone, gst, email, password
2
Preeti
8521479630
PRTG1234
preeti@gmail.com
preeti4321
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Hibernate:
    update
        Merchant
    set
        email=?,
        gst_num=?,
        name=?,
        password=?,
        phone=?
    where
        id=?
Merchant info is updated!!
```

After Update :

id	email	gst_num	name	password	phone
1	ganesh@gmail.com	GKM12345	Ganesh	ganesh12345	8855221144
2	preeti@gmail.com	PRTG1234	Preeti	preeti4321	8521479630
3	suman@gmail.com	SUM12334	Suman	suman12345	8855336622
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Case 3 : Find Merchant by Id

MerchantProductController.java

```
private static void findMerchantById() {
    System.out.println("Enter Merchant id to find??");
    int mid = sc.nextInt();
    Merchant mdb = mdao.findMerchantById(mid);
    if(mdb != null)
    {
        System.out.println(mdb);
    }
    else {
        System.out.println("Merchant info not found since Merchant id is invalid!!");
    }
}
```

MerchantDao.java

```
    public Merchant findMerchantById(int mid) {
        return man.find(Merchant.class, mid);
    }
```

Output :

```
Enter Your choice??
3
Enter Merchant id to find??
1
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Merchant [id=1, name=Ganesh, phone=8855221144, gst_num=GKM12345, email=ganesh@gmail.com, password=ganesh12345]
```

Case 4 : Verify Merchant by Email and Password

MerchantProductController.java

```
private static void verifyMerchantByEmailAndPassword() {
    System.out.println("Enter Email??");
    String em = sc.next();
    System.out.println("Enter Password??");
    String pw = sc.next();

    Merchant mdb = mdao.verifyMerchantByEmailAndPassword(em, pw);
    if(mdb != null)
    {
        System.out.println("Merchant is verified!");
    }
    else {
        System.out.println("Merchant is not Verified!!");
    }
}
```

MerchantDao.java

```
public Merchant verifyMerchantByEmailAndPassword(String em, String pw) {
    Query q = man.createQuery("select m from Merchant m where m.email=?1 and m.password=?2");
    q.setParameter(1, em);
    q.setParameter(2, pw);
    try {
        Merchant m = (Merchant) q.getSingleResult();
        return m;
    } catch (NoResultException e) {
        return null;
    }
}
```

Output :

```
Enter Your choice??
4
Enter Email??
preeti@gmail.com
Enter Password??
preeti4321
Hibernate:
    select
        merchant0_.id as id1_0_,
        merchant0_.email as email2_0_,
        merchant0_.gst_num as gst_num3_0_,
        merchant0_.name as name4_0_,
        merchant0_.password as password5_0_,
        merchant0_.phone as phone6_0_
    from
        Merchant merchant0_
    where
        merchant0_.email=?
        and merchant0_.password=?
Merchant is verified!
```

Case 5 : Verify Merchant by Phone and Password

MerchantProductController.java

```
private static void verifyMerchantByPhoneAndPassword() {
    System.out.println("Enter Phone ??");
    long ph = sc.nextLong();
    System.out.println("Enter Password??");
    String pw = sc.next();

    Merchant mdb = mdao.verifyMerchantByPhoneAndPassword(ph, pw);
    if(mdb != null)
    {
        System.out.println("Merrchant is Verified!");
    }
    else {
        System.out.println("Merchant is not Verified!");
    }
}
```

MerchantDao.java

```
public Merchant verifyMerchantByPhoneAndPassword(long ph, String pw) {
    Query q = man.createQuery("select m from Merchant m where m.phone=?1 and m.password=?2");
    q.setParameter(1, ph);
    q.setParameter(2, pw);
    try {
        Merchant m = (Merchant) q.getSingleResult();
        return m;
    } catch(NoResultException e)
    {
        return null;
    }
}
```

Output :

```
Enter Your choice??
5
Enter Phone ??
8855221144
Enter Password??
ganesh12345
Hibernate:
    select
        merchant0_.id as id1_0_,
        merchant0_.email as email2_0_,
        merchant0_.gst_num as gst_num3_0_,
        merchant0_.name as name4_0_,
        merchant0_.password as password5_0_,
        merchant0_.phone as phone6_0_
    from
        Merchant merchant0_
    where
        merchant0_.phone=?
        and merchant0_.password=?
Merrchant is Verified!
```

Case 6 : Add Product

MerchantProductController.java

```
private static void addProduct() {
    System.out.println("Enter Merchant id to find Merchant");
    int mid = sc.nextInt();

    System.out.println("Enter the Product info ----name, brand, category, cost");
    Product p = new Product();
    p.setName(sc.next());
    p.setBrand(sc.next());
    p.setCategory(sc.next());
    p.setCost(sc.nextDouble());
    Product pdb = pdao.addProduct(mid, p);
    if(pdb != null)
    {
        System.out.println("Product is added to the Merchant with an id " + pdb.getId());
    }
    else {
        System.out.println("Unable to add the product since mod is invalid!!");
    }
}
```

ProductDao.java

```
import java.util.List;
import javax.persistence.*;
import org.jsp.merchantapp.*;

public class ProductDao {
    EntityManagerFactory fac = Persistence.createEntityManagerFactory("dev");
    EntityManager man = fac.createEntityManager();

    public Product addProduct(int mid, Product p) {
        EntityTransaction tran = man.getTransaction();
        tran.begin();
        Merchant mdb = man.find(Merchant.class, mid);
        if(mdb != null)
        {
            p.setMerchant(mdb);
            mdb.getProducts().add(p);
            man.persist(p);
            tran.commit();
            return p;
        }
        else {
            return null;
        }
    }
```

Output :

```
Enter Your choice??
6
Enter Merchant id to find Merchant
2
Enter the Product info ----name, brand, category, cost
Camera
Sony
Electronic
1000000
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Hibernate:
    insert
    into
        Product
        (brand, category, cost, merchant_id, name)
    values
        (?, ?, ?, ?, ?)
Product is added to the Merchant with an id 2
```

Table :

Query | Table Data

id	brand	category	cost	name	merchant_id
1	Sony	Electronics	350000	TV	1
2	Sony	Electronic	1000000	Camera	2
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Case 7 : Update Product

MerchantProductController.java

```
private static void updateProduct() {
    System.out.println("Enter Product Info to Update... id, name, brand, category, cost");
    Product p = new Product();
    p.setId(sc.nextInt());;
    p.setName(sc.next());
    p.setBrand(sc.next());
    p.setCategory(sc.next());
    p.setCost(sc.nextDouble());
    Product pdb = pdao.updateProduct(p);
    if(pdb != null)
    {
        System.out.println("Product info is updated!!");
    }
    else {
        System.out.println("Product info is not updated!");
    }
}
```

ProductDao.java

```
public Product updateProduct(Product p) {
    EntityTransaction tran = man.getTransaction();
    tran.begin();
    Product pdb = man.find(Product.class, p.getId());
    if(pdb != null)
    {
        pdb.setName(p.getName());
        pdb.setBrand(p.getBrand());
        pdb.setCategory(p.getCategory());
        pdb.setCost(p.getCost());
        tran.commit();
        return p;
    }
    else {
        return null;
    }
}
```

Before Table :

Query | Table Data

id	brand	category	cost	name	merchant_id
1	Sony	Electronics	350000	TV	1
2	Sony	Electronic	1000000	Camera	2
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Output :

```
Enter Product Info to Update... id, name, brand, category, cost
2
Bluetooth
OnePlus
Electronics
40000
Hibernate:
    select
        product0_.id as id1_1_0_,
        product0_.brand as brand2_1_0_,
        product0_.category as category3_1_0_,
        product0_.cost as cost4_1_0_,
        product0_.merchant_id as merchant6_1_0_,
        product0_.name as name5_1_0_,
        merchant1_.id as id1_0_1_,
        merchant1_.email as email2_0_1_,
        merchant1_.gst_num as gst_num3_0_1_,
        merchant1_.name as name4_0_1_,
        merchant1_.password as password5_0_1_,
        merchant1_.phone as phone6_0_1_
    from
        Product product0_
    left outer join
        Merchant merchant1_
            on product0_.merchant_id=merchant1_.id
    where
        product0_.id=?
Hibernate:
    update
        Product
    set
        brand=?,
        category=?,
        cost=?,
        merchant_id=?,
        name=?
    where
        id=?
Product info is updated!!
```

Output :

Query | Table Data

id	brand	category	cost	name	merchant_id
1	Sony	Electronics	350000	TV	1
2	OnePlus	Electronics	40000	Bluetooth	2
(Auto)	(NULL)	(NULL)	(NULL)	(NULL)	(NULL)

Case 8 : Find Products by Merchant id

MerchantProductController.java

```
private static void findProductByMerchantId() {
    System.out.println("Enter Merchant id ??");
    int mid = sc.nextInt();

    List<Product> lpdb = pdao.findProductByMerchantId(mid);
    if(lpdb.size() > 0)
    {
        for(Product product : lpdb)
        {
            System.out.println(product);
        }
    }
    else {
        System.out.println("No Product info found since Merchant id is Invalid!");
    }
}
```

ProductDao.java

```
public List<Product> findProductByMerchantId(int mid) {
    Query q = man.createQuery("select m.products from Merchant m where m.id=?1");
    q.setParameter(1, mid);
    List<Product> lp= q.getResultList();
    return lp;
}
```

Output :

```
Enter Your choice??
8
Enter Merchant id ??
2
Hibernate:
    select
        products1_.id as id1_1_,
        products1_.brand as brand2_1_,
        products1_.category as category3_1_,
        products1_.cost as cost4_1_,
        products1_.merchant_id as merchant6_1_,
        products1_.name as name5_1_
    from
        Merchant merchant0_
    inner join
        Product products1_
            on merchant0_.id=products1_.merchant_id
    where
        merchant0_.id=?
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Product [id=2, name=Bluetooth, brand=OnePlus, category=Electronics, cost=40000.0]
```

Case 9 : Find products by brand

MerchantProductController.java

```
private static void findProductByBrand() {
    System.out.println("Enter Brand");
    String brand = sc.next();
    List<Product> lpdb = pdao.findProductByBrand(brand);
    if(lpdb != null)
    {
        for(Product product : lpdb)
        {
            System.out.println(product);
        }
    }
    else {
        System.out.println("No Product info found since brand is invalid!");
    }
}
```

ProductDao.java

```
public List<Product> findProductByBrand(String brand) {
    Query q = man.createQuery("select p from Product p where p.brand=?1");
    q.setParameter(1, brand);
    List<Product> lp = q.getResultList();
    return lp;

}
```

Output :

```
Enter Your choice??
9
Enter Brand
Sony
Hibernate:
    select
        product0_.id as id1_1_,
        product0_.brand as brand2_1_,
        product0_.category as category3_1_,
        product0_.cost as cost4_1_,
        product0_.merchant_id as merchant6_1_,
        product0_.name as name5_1_
    from
        Product product0_
    where
        product0_.brand=?
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Product [id=1, name=TV, brand=Sony, category=Electronics, cost=350000.0]
```

Case 10 : Find products by Category

MerchantProductController.java

```
private static void findProductByCategory() {
    System.out.println("Enter Category");
    String category = sc.next();
    List<Product> lpdb = pdao.findProductByCategogy(category);
    if(lpdb != null)
    {
        for(Product product : lpdb)
        {
            System.out.println(product);
        }
    }
    else {
        System.out.println("No Product info found since category is invalid!");
    }
}
```

ProductDao.java

```
public List<Product> findProductByCategogy(String category) {
    Query q = man.createQuery("select p from Product p where p.category=?1");
    q.setParameter(1, category);
    List<Product> lp = q.getResultList();
    return lp;
}
```

Output :

```
Enter Your choice??
10
Enter Category
Electronics
Hibernate:
    select
        product0_.id as id1_1_,
        product0_.brand as brand2_1_,
        product0_.category as category3_1_,
        product0_.cost as cost4_1_,
        product0_.merchant_id as merchant6_1_,
        product0_.name as name5_1_
    from
        Product product0_
    where
        product0_.category=?
Hibernate:
    select
        merchant0_.id as id1_0_0_,
        merchant0_.email as email2_0_0_,
        merchant0_.gst_num as gst_num3_0_0_,
        merchant0_.name as name4_0_0_,
        merchant0_.password as password5_0_0_,
        merchant0_.phone as phone6_0_0_
    from
        Merchant merchant0_
    where
        merchant0_.id=?
Product [id=2, name=Bluetooth, brand=OnePlus, category=Electronics, cost=40000.0]
```

If Default :

```
Enter Your choice??
15
Guldu!!!, Open your eyes and see only 10 Choices are there
```

6. SUMMARY

The **Merchant Product Management System using Hibernate & MySQL** is a Java-based application developed to manage merchant and product information in a simple, secure, and efficient manner. The project was designed to replace manual record maintenance with a computerized system that stores data in a structured database. It helps users perform important operations such as merchant registration, merchant verification, adding products, updating records, and searching products based on different criteria.

The system consists of two main modules: Merchant Module and Product Module. The Merchant Module handles merchant details such as name, phone number, GST number, email, and password. The Product Module manages product details such as product name, brand, category, and cost. Both modules are connected through a One-to-Many relationship where one merchant can own multiple products.

The project was implemented using **Java** as the programming language, **Hibernate ORM** for database mapping, and **MySQL** for data storage. Hibernate simplified database operations such as insert, update, delete, and fetch by converting Java objects into database records. This reduced manual SQL coding and improved code maintainability.

The system provides many advantages such as reduced paperwork, improved data accuracy, fast data retrieval, secure login verification, and better management of merchant and product records. It also offers practical knowledge of object-oriented programming, database connectivity, Hibernate annotations, entity relationships, and CRUD operations.

This project is highly useful for students and beginners who want to understand real-time Java application development using Hibernate and MySQL. It can be further enhanced by adding a graphical user interface, web version, admin dashboard, billing system, stock management, and online payment features.

In conclusion, the Merchant Product Management System successfully demonstrates how software technology can be used to manage business records efficiently. It is a valuable academic project as well as a strong foundation for future real-time business management applications.